

AI Asystent regulaminów

Aplikacja pełniąc rolę asystenta AI odpowiadającego na pytania dotyczące regulaminów i zarządzeń obowiązujących w Instytucie Historii PAN. Teksty dokumentów pochodzą z plików PDF i DOCX udostępnionych pracownikom Instytutu; część skanów została automatycznie przetworzona przez OCR, więc w tekstach mogą występować literówki i błędy odczytu.

Główne elementy:

- `app.py` - aplikacja Flask, routing, logowanie, API chatu, API dokumentów i wyszukiwanie pełnotekstowe.
- `rag.py` - lokalna baza wiedzy z plików Markdown i wyszukiwarka fragmentów.
- `llm.py` - budowanie kontekstu, wywołania Gemini i wybór tematów wiki dla pytania.
- `build_wiki.py` - generator tematycznej struktury `wiki/`.
- `templates/index.html` - główny widok chatu.
- `templates/documents.html` - widok przeglądania i wyszukiwania dokumentów.
- `static/app.js` - obsługa interfejsu w przeglądarce.
- `md/` - katalog źródłowych dokumentów Markdown, z których budowany jest indeks.
- `wiki/` - automatycznie generowana mapa tematyczna dokumentów, używana w widoku dokumentów i jako pomocnicza preferencja przy wyszukiwaniu dla chatu.

Aplikacja nie używa wektorowej bazy danych ani embeddingów. Podstawowe wyszukiwanie jest lokalne i leksykalne: dokumenty są dzielone na fragmenty, tokenizowane, a następnie oceniane na podstawie dopasowania słów z pytania do słów we fragmentach. Dodatkowo chat może użyć modelu językowego do rozpoznania, czy pytanie szczególnie pasuje do któregoś tematu z `wiki/`; dokumenty z takiego tematu dostają wtedy dodatkową preferencję w rankingu.

Start Aplikacji

Przy imporcie `app.py` tworzona jest aplikacja Flask i ustawiany jest `ProxyFix`, żeby aplikacja poprawnie działała za proxy, np. z `gunicornem/nginxem`.

Najważniejsze jest utworzenie globalnego obiektu bazy wiedzy:

```
kb = KnowledgeBase(MD_DIR)
```

Konstruktor `KnowledgeBase` od razu wywołuje `reload()`. W efekcie przy starcie aplikacji:

1. Czyszczona jest lista dokumentów i fragmentów.
2. Aplikacja przechodzi po plikach `*.md` w katalogu `MD_DIR`.
3. Każdy plik jest czytany i dzielony na fragmenty przez `split_document()`.

4. Dla każdego fragmentu tworzone są tokeny używane później w wyszukiwaniu.

Indeks jest więc budowany w pamięci przy starcie aplikacji, a nie przy każdym pytaniu.

Konfiguracja

Konfiguracja znajduje się w `config.py`.

Domyslnie:

- dokumenty są czytane z `md/`,
- pomocnicza wiki jest zapisywana w `wiki/`,
- tytuł aplikacji to **Asystent regulaminów i zarządzeń Instytutu Historii PAN**,
- model Gemini to `gemini-3-flash-preview`,
- limit wyszukiwanych fragmentów to `SEARCH_LIMIT = 8`,
- maksymalna długość kontekstu dla modelu to `MAX_CONTEXT_CHARS = 18000`,
- premia dla dokumentów z tematu wiki wynosi `WIKI_TOPIC_BONUS = 0.45`.

Plik `.env`, jeśli istnieje, jest ładowany ręcznie przez `load_env_file()`. Klucz do Gemini jest brany z `GOOGLE_API_KEY` albo `GEMINI_API_KEY`.

Rola Katalogu wiki/

Katalog `wiki/` jest warstwą pomocniczą zbudowaną na podstawie dokumentów z `md/`. Nie zastępuje on źródłowych dokumentów: właściwa treść odpowiedzi i cytowane źródła nadal pochodzą z plików Markdown w `md/`.

`wiki/` pełni obecnie dwie role:

1. W widoku dokumentów pozwala przeglądać dokumenty tematycznie.
2. W chacie pomaga wstępnie wskazać tematy, które mogą pasować do pytania użytkownika.

Najważniejsze pliki w `wiki/`:

- `wiki/index.md` - indeks tematów oraz statystyki liczby dokumentów i fragmentów.
- `wiki/<temat>.md` - lista dokumentów przypisanych do tematu, z datą, ścieżką, tytułem i oznaczeniem zmian/aneksów/uchyleń.

Tematy są definiowane w `build_wiki.py`. Generator korzysta głównie z reguł opartych o nazwy plików, tytuły i słowa kluczowe. Tematy w indeksie są sortowane alfabetycznie, a dokumenty w obrębie tematów chronologicznie od najnowszych.

Do przebudowania tej warstwy służy:

```
python manage_index.py build-wiki
```

albo równoważnie:

```
python manage_index.py refresh
```

Logowanie

Każde żądanie przechodzi przez `require_login()` w `app.py`.

Jeśli użytkownik nie jest zalogowany:

- dla stron HTML dostaje przekierowanie na `/login`,
- dla endpointów `/api/...` dostaje JSON z błędem logowania.

Logowanie porównuje login i hasło z konfiguracji przez `hmac.compare_digest()`. Po sukcesie ustawia w sesji:

```
session["authenticated"] = True
```

Ekran Główny

Po wejściu na `/` renderowany jest główny widok chatu z `templates/index.html`.

Na stronie jest:

- nagłówek z tytułem,
- liczba dokumentów,
- ostrzeżenie, że AI może się mylić,
- obszar wiadomości `#messages`,
- formularz `#chat-form`,
- textarea `#message`,
- przycisk Dokumenty,
- przycisk Info,
- modal do podglądu dokumentów źródłowych.

Przycisk **Info** otwiera okno modalne z krótkim opisem aplikacji, pochodzenia dokumentów, ograniczeń OCR oraz dwóch głównych modułów: chatu i dokumentów.

Widok Dokumentów

Widok `/documents` pokazuje dokumenty w trzech trybach przełączanych zakładkami:

- **Chronologicznie** - dotychczasowa lista dokumentów, sortowana od najnowszych.
- **Tematycznie** - lista tematów z `wiki/index.md`, które można rozwijać strzałką; pod każdym tematem są dokumenty przypisane do tego tematu, również sortowane od najnowszych.
- **Wyszukiwanie** - pełnotekstowe wyszukiwanie w dokumentach Markdown.

Wyszukiwanie pełnotekstowe działa przez endpoint:

POST /api/document-search

Frontend wysyła szukaną frazę, a backend przegląda pełną treść dokumentów z md/. Wyniki są pokazywane jako boksy podobne do zwykłej listy dokumentów, ale rozszerzone o:

- liczbę trafień w dokumencie,
- fragment tekstu z pierwszym trafieniem,
- podświetlenie szukanego słowa lub wyrażenia na żółto,
- informację o pliku i numerze linii.

Przycisk **Wyczyść** resetuje pole wyszukiwania, komunikat stanu i listę wyników.

Proces Pytania w Chacie

Najważniejsza ścieżka zaczyna się w static/app.js. Frontend nasłuchuje wysłania formularza, czyści tekst pytania i wysyła JSON:

```
{  
  "message": "pytanie użytkownika"  
}
```

do endpointu:

POST /api/chat

W czasie oczekiwania w interfejsie pojawia się tymczasowy komunikat Szukam w dokumentach....

Endpoint /api/chat

Backend odbiera pytanie, waliduje je, a potem wykonuje procedurę RAG:

1. Czyta tematy z wiki/index.md i powiązane dokumenty z plików wiki/<temat>.md.
2. Wywołuje select_wiki_topics(), aby model wskazał zero, jeden albo kilka tematów szczególnie pasujących do pytania.
3. Zamienia wybrane tematy na zbiór ścieżek dokumentów preferowanych.
4. Wywołuje kb.search() z pytaniem, limitem wyników oraz preferowanymi dokumentami.
5. Nadaje fragmentom identyfikatory źródeł S1, S2, S3 itd.
6. Buduje kontekst dla Gemini.
7. Generuje odpowiedź wyłącznie na podstawie przekazanych fragmentów.
8. Wykrywa, które źródła model zacytował.
9. Zwraca JSON z odpowiedzią, źródłami i statystykami indeksu.

Wybór tematów wiki jest miękką preferencją, a nie filtrem. Jeśli model uzna, że żaden temat nie pasuje wyraźnie do pytania, zwraca pustą listę i wyszukiwanie działa tak jak zwykle wyszukiwanie leksykalne. Jeśli brakuje klucza API albo

wywołanie modelu się nie powiedzie, mechanizm tematyczny również przechodzi do pustego wyboru.

Wyszukiwanie Fragmentów

`kb.search()` w `rag.py` tokenizuje pytanie:

- wyszukuje słowa regexem,
- stosuje `casefold()`, czyli ujednolica wielkość liter,
- usuwa krótkie słowa,
- usuwa stopwords typu `i`, `w`, `do`, `oraz`, `nie`, `się`,
- dodaje wybrane rozszerzenia zapytań, np. dla form związanych z mobbingiem i procedurą antymobbingową.

Następnie każdy fragment dokumentu jest oceniany. Podstawą score jest wspólna część tokenów pytania i tokenów fragmentu. Do tego dochodzą m.in.:

- premia za występowanie fraz z pytania w tekście,
- niewielka premia za nowsze dokumenty,
- premia dla dokumentów wyglądających na zmianę/aneks/uchylenie,
- premia `WIKI_TOPIC_BONUS` dla dokumentów należących do tematów wybranych przez model.

Jeśli dokument pochodzi z wybranego tematu wiki, ale dany fragment nie ma bezpośredniego dopasowania leksykalnego, może nadal otrzymać małą część premii tematycznej. To pomaga w sytuacjach, gdy użytkownik opisuje problem innymi słowami niż te, które dosłownie występują w dokumencie.

Do `/api/chat` wraca maksymalnie `SEARCH_LIMIT` fragmentów, domyślnie 8.

Jak Powstają Fragmenty

Fragmenty powstają przy starcie aplikacji w `split_document()`.

Dla każdego pliku Markdown aplikacja:

1. Wyciąga tytuł z pierwszego nagłówka Markdown w pierwszych liniach pliku.
2. Próbuje znaleźć datę w nazwie pliku albo początku tekstu.
3. Rozpoznaje typ dokumentu po słowach typu `zarządzenie`, `regulamin`, `instrukcja`, `aneks`.
4. Sprawdza, czy dokument wygląda na zmianę/aneks/uchylenie.
5. Dzieli tekst na chunki po nagłówkach Markdown albo gdy fragment przekroczy ustalony rozmiar.

Każdy chunk ma m.in. `path`, `title`, `date`, `heading`, `start_line`, `end_line`, `text` i `is_change`. Dzięki temu aplikacja może później pokazać źródło oraz konkretne linie w dokumencie.

Budowanie Kontekstu dla Gemini

Po wyszukaniu fragmentów backend wywołuje `attach_source_ids()`, które nadaje kolejnym fragmentom identyfikatory:

S1, S2, S3, ...

Te identyfikatory są przekazywane modelowi i używane w cytowaniach.

Kontekst dla Gemini zawiera przy każdym źródle:

- identyfikator źródła,
- tytuł,
- datę,
- ścieżkę pliku,
- sekcję,
- zakres linii,
- informację, czy dokument wygląda na zmianę/aneks/uchylenie,
- treść fragmentu.

Kontekst jest dokładany tylko do limitu `MAX_CONTEXT_CHARS`.

Instrukcja systemowa mówi modelowi m.in., aby:

- odpowiadał jako asystent IH PAN,
- odpowiadał wyłącznie na podstawie przekazanych fragmentów,
- jeśli fragmenty nie wystarczają, powiedział to wprost,
- każde twierdzenie o treści dokumentów cytował jako [S1], [S2],
- zaznaczał, jeśli źródło wygląda na aneks, zmianę albo uchylenie,
- nie udzielał porady prawnej,
- odpowiadał po polsku, rzeczowo i zwięźle.

Fallback Bez Gemini

Jeśli nie ma klucza API albo wywołanie Gemini rzuci wyjątek, działa fallback. Nie generuje on właściwej odpowiedzi modelowej, tylko pokazuje najlepsze znalezione źródła z krótkimi fragmentami. Jeśli nie znaleziono żadnych fragmentów, zwracany jest komunikat, że nie ma podstaw do odpowiedzi.

Wyświetlanie Odpowiedzi i Źródeł

Frontend odbiera odpowiedź, zastępuje komunikat `Szukam w dokumentach...` treścią odpowiedzi i renderuje ją jako prosty Markdown. Lokalny renderer obsługuje m.in. nagłówki, listy, inline code, pogrubienie i kursywę. Tekst jest escapowany przed renderowaniem elementów inline.

Pod odpowiedzią pojawia się rozwijana lista źródeł. Każde źródło pokazuje:

- identyfikator [S1],
- tytuł dokumentu,
- datę,

- nagłówek lub sekcję,
- zakres linii,
- przycisk ze ścieżką pliku.

Kliknięcie źródła otwiera pełny dokument w modalu i podświetla linie, z których pochodził fragment.

Podgląd i Wyszukiwanie w Dokumentcie

Dokument źródłowy jest pobierany przez:

```
GET /api/document?path=...
```

Backend zabezpiecza ten endpoint przez użycie samej nazwy pliku, dołączenie jej do MD_DIR, rozwiązanie ścieżki i sprawdzenie, czy finalny plik znajduje się bezpośrednio w katalogu md/ oraz ma rozszerzenie .md.

Modal dokumentu ma własne pole wyszukiwania. Po wpisaniu frazy:

- wszystkie trafienia w dokumentcie są podświetlane na zielono,
- aktywne trafienie jest podświetlane na pomarańczowo,
- licznik pokazuje pozycję typu 1/12,
- przyciski < i > przechodzą do poprzedniego lub następnego trafienia,
- Enter przechodzi do następnego trafienia,
- Shift+Enter przechodzi do poprzedniego trafienia.

Enter i Shift+Enter w Chacie

Textarea chatu ma osobną obsługę klawiatury:

- Enter wysyła formularz,
- Shift+Enter pozwala wpisać nową linię,
- puste pytanie nie jest wysyłane.

Najważniejsza Sekwencja w Skrócie

1. Użytkownik wpisuje pytanie w textarea.
2. Frontend wysyła POST /api/chat z { message }.
3. Flask sprawdza sesję logowania.
4. Backend czyta strukturę wiki/.
5. Model wybiera pasujące tematy wiki albo zwraca pustą listę.
6. Dokumenty z wybranych tematów dostają preferencję w rankingu.
7. kb.search() znajduje pasujące fragmenty Markdown.
8. attach_source_ids() dodaje źródła S1, S2, ...
9. Gemini generuje odpowiedź na podstawie przekazanego kontekstu.
10. cited_sources() wykrywa, które źródła model zacytował.
11. Frontend renderuje odpowiedź i listę źródeł.
12. Kliknięcie źródła otwiera pełny dokument w modalu.

Istotne Ograniczenia

Aplikacja nadal opiera się głównie na wyszukiwaniu leksykalnym. Mechanizm wyboru tematów wiki poprawia dobór dokumentów w części przypadków, ale nie jest pełnym wyszukiwaniem semantycznym i nie gwarantuje znalezienia wszystkich właściwych źródeł.

Model nie widzi całego katalogu dokumentów, tylko fragmenty wybrane przez wyszukiwarkę. Jeśli wyszukiwarka nie wybierze właściwego fragmentu, Gemini nie ma jak poprawnie odpowiedzieć na podstawie tego dokumentu.

Wybór tematów wiki wymaga dodatkowego wywołania modelu, więc może zwiększać czas odpowiedzi i koszt użycia API.

Struktura `wiki/` jest generowana automatycznie i wymaga utrzymywania reguł w `build_wiki.py`. Błędne albo zbyt szerokie przypisanie dokumentu do tematu może wpływać na kolejność wyników, choć nie zmienia treści dokumentów ani cytowanych źródeł.

Pełnotekstowe wyszukiwanie w widoku dokumentów działa po dosłownej frazie lub słowie w tekście Markdown. Nie wykonuje odmiany fleksyjnej, wyszukiwania semantycznego ani korekty literówek.

Źródła pokazywane pod odpowiedzią zależą od tego, czy model użył cytowań `S1`, `S2` itd. Instrukcja systemowa tego wymaga, ale kod nie waliduje twardo, czy każde twierdzenie faktycznie ma cytowanie.

Aplikacja ładuje indeks przy starcie. Jeśli ktoś zmieni pliki w `md/` podczas działania serwera, globalne `kb` samo się nie przeładuje, chyba że aplikacja zostanie zrestartowana albo kod zostanie rozszerzony o ręczne `kb.reload()`.